PRACTICAL - 03

Part A — Practice Programs (Hour 1)

PROBLEM1	**1. Distinct points (Point.java + Driver.java with main)** **(a) Create class Point with private int x, y and a constructor.** **(b) Override toString() to return "(x, y)".** **(c) Override equals(Object) to be true when both x and y match, and hashCode() (e.g. Objects.hash(x, y)).** **(d) In Driver.main: build a Point[] with some repeated coordinates.** **(e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not.** **(f) Print "Distinct: N".** **Expected output: 5 points with 2 repeats → "Distinct: 3".**

CODE	public class Point {
	  private int x, y;
	
	  public Point(int x, int y) {
	  this.x = x;
	  this.y = y;
	  }
	
	  public void print() {
	  System.out.print("(" + x + ", " + y + ")");
	  }
	
	  public boolean isSame(Point other) {
	  return this.x == other.x && this.y == other.y;
	  }
	}
	
	public class Driver {
	public static void main(String[] args) {
	  Point[] pts = {
	  new Point(1, 2),
	  new Point(3, 4),
	  new Point(1, 2),
	  new Point(5, 6),
	  new Point(3, 4)
	  };
	
	  int count = 0;
	  for (int i = 0; i < pts.length; i++) {
	  boolean seen = false;
	  for (int j = 0; j < i; j++) {
	  if (pts[i].isSame(pts[j])) {

```
            seen = true;
            break;
          }
        }
        if (!seen) count++;
      }
      System.out.println("Distinct: " + count);
    }
}
```

OUTPUT

```
[Done] exited with code=0 in 0.791 seconds

[Running] cd "c:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 3\" && javac Driver.java && java Driver
Number of distinct points: 1

[Done] exited with code=0 in 0.716 seconds
```

PROBLEM2

2. Duplicate card check (Card.java + Driver.java)
(a) Create class Card with private String rank, suit and a constructor.
(b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").
(c) Override equals()/hashCode() by rank + suit.
(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards.
(e) Print "Duplicate found: <card>" the first time a repeat appears.
Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades".

CODE

```
public class Card {
  private String rank, suit;

  public Card(String rank, String suit) {
    this.rank = rank;
    this.suit = suit;
  }

  public void print() {
    System.out.print(rank + " of " + suit);
  }

  public boolean isSame(Card other) {
    return this.rank.equals(other.rank) && this.suit.equals(other.suit);
  }
}

public class CardTest {
  public static void main(String[] args) {
    Card[] cards = new Card[5];
    int size = 0;
```

```
        Card[] newCards = {
            new Card("Ace", "Spades"),
            new Card("King", "Hearts"),
            new Card("Queen", "Hearts"),
            new Card("Ace", "Spades"),
            new Card("Jack", "Clubs")
        };

        for (Card c : newCards) {
            boolean dup = false;
            for (int i = 0; i < size; i++) {
                if (cards[i].isSame(c)) {
                    dup = true;
                    break;
                }
            }
            if (dup) {
                System.out.print("Duplicate found: ");
                c.print();
                System.out.println();
                break;
            }
            cards[size] = c;
            size++;
        }
    }
}
```

OUTPUT

```
[Done] exited with code=0 in 0.716 seconds

[Running] cd "c:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 3\" && javac CardTest.java && java CardTest
Duplicate found: Ace of Spades

[Done] exited with code=0 in 0.708 seconds
```

PROBLEM3

3. (For Advanced Learners – home) Fractions (Fraction.java + Driver.java)

(a) Create class Fraction with private int num, den.

(b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g.

(c) Override toString() to return num + "/" + den.

(d) Override equals()/hashCode() using the reduced num and den.

(e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.

Expected output: new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).

CODE

```
public class Fraction {
   private int num, den;

   public Fraction(int num, int den) {
      int g = gcd(Math.abs(num), Math.abs(den));
      this.num = num / g;
      this.den = den / g;
   }

   private int gcd(int a, int b) {
      if (b == 0) return a;
      return gcd(b, a % b);
   }

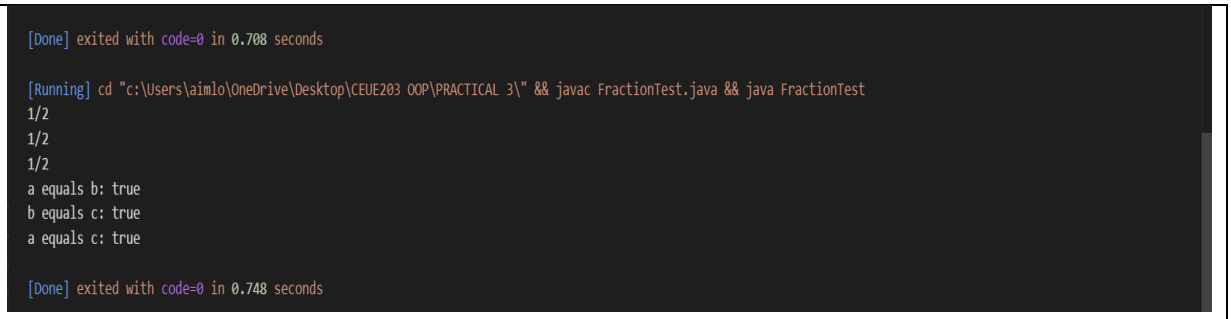
   public void print() {
      System.out.print(num + "/" + den);
   }

   public boolean isSame(Fraction other) {
      return this.num == other.num && this.den == other.den;
   }
}

public class FractionTest {
   public static void main(String[] args) {
      Fraction a = new Fraction(1, 2);
      Fraction b = new Fraction(2, 4);
      Fraction c = new Fraction(3, 6);

      a.print();
      System.out.println();
      b.print();
      System.out.println();
      c.print();
      System.out.println();

      System.out.println("a equals b: " + a.isSame(b));
      System.out.println("b equals c: " + b.isSame(c));
      System.out.println("a equals c: " + a.isSame(c));
   }
}
```

OUTPUT	

Part B — Project Work (Hour 2): Mini Bank

PROBLEM	In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it. 1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance. 2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal. 3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method. 4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer. 5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type.

CODE	public class MiniBank {
	
	public static void main(String[] args) throws CloneNotSupportedException {
	
	Customer.Address addr = new Customer.Address(
	"Atmiya Society",
	"Vadodara",
	"390001");
	
	Customer c = new Customer(
	"Dharmik Sonani",
	"dharmik@gmail.com",
	"9930030060",
	addr);
	
	Customer c2 = c.clone();
	
	Account[] acc = new Account[3];

```
        acc[0] = new Account(c.getName(), 5000);
        acc[1] = new Account("Bhavy", 3000);
        acc[2] = new Account("Manthan");

        acc[0].deposit(1000);
        acc[0].withdraw(2000);

        acc[1].deposit(500);
        acc[1].withdraw(1000);

        acc[2].deposit(2000);
        acc[2].withdraw(500);

        System.out.println("Accounts:");

        for (Account a : acc) {
            System.out.println(a.toString());
        }

        System.out.println();

        System.out.println("Comparing Accounts:");
        System.out.println(acc[0].equals(acc[1]));
        System.out.println(acc[0].equals(acc[0]));

        Object obj = acc[0];

        if (obj instanceof Account) {
            System.out.println("obj is an Account object.");
        }

        if (c2 instanceof Customer) {
            System.out.println("c2 is a Customer object.");
        }
    }
}

class Customer implements Cloneable {

    private String name;
    private String email;
    private String phone;
    private final String id;
    private Address addr;

    private static long count = 101;
```

```
    private static String createId() {
        return "CUST" + count++;
    }

    public Customer(String name, String email, String phone, Address addr) {
        this.name = name;
        this.email = email;
        this.phone = phone;
        this.addr = addr;
        this.id = createId();
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getPhone() {
        return phone;
    }

    public String getId() {
        return id;
    }

    public Address address() {
        return addr;
    }

    public Customer clone() throws CloneNotSupportedException {
        return (Customer) super.clone();
    }

    public static class Address {

        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }
```

```
        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }
    }
}

class Account {

    private final String accNo;
    private String owner;
    private long bal;
    private boolean active;

    private static int count = 1;

    private static String createAccNo() {
        return String.format("AC%04d", count++);
    }

    public Account(String owner, long bal) {
        this.accNo = createAccNo();
        this.owner = owner;
        this.bal = bal;
        this.active = true;
    }

    public Account(String owner) {
        this(owner, 0);
    }

    public void deposit(long amount) {
        if (amount < 0)
            return;

        bal += amount;
    }

    public boolean withdraw(long amount) {
```

```
    if (amount < 0)
        return false;

    if (bal >= amount) {
        bal -= amount;
        return true;
    }

    return false;
}

public String getAccNo() {
    return accNo;
}

public String getOwner() {
    return owner;
}

public long getBal() {
    return bal;
}

public boolean isActive() {
    return active;
}

public String toString() {
    return accNo + " " + owner + " Balance: " + bal;
}

public boolean equals(Object o) {

    if (this == o)
        return true;

    if (!(o instanceof Account))
        return false;

    Account a = (Account) o;

    return accNo.equals(a.accNo);
}

public int hashCode() {
    return accNo.hashCode();
```

```
    }
}
```

OUTPUT

```
[Running] cd "c:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 3\" && javac MiniBank.java && java MiniBank
Accounts:
AC0001 Om Sutariya Balance: 4000
AC0002 Bhavy Balance: 2500
AC0003 Manthan Balance: 1500

Comparing Accounts:
false
true
obj is an Account object.
c2 is a Customer object.

[Done] exited with code=0 in 0.816 seconds
```